

Linux Incident Response Evidence Collector

Team Members:

- Logan Flowers
- Hunter Mayes
- Brett Kelly

Introduction:

The purpose of this project is to create a Linux Incident Response Evidence Collector that helps investigators quickly gather important forensic information from a Linux system after a security incident. During an incident response investigation, analysts often need to manually collect system information, running processes, network connections, log files, and other evidence. This process can be time-consuming, inconsistent, and may lead to important evidence being missed.

Our project solves this problem by automating the evidence collection process with a Bash script. The script gathers important system data, stores it in an organized evidence folder, hashes the collected artifacts to preserve integrity, and compresses and encrypts the final evidence package. This allows investigators to collect information quickly while maintaining forensic principles such as data integrity, repeatability, and chain of custody.

The script is designed to work on Linux systems and is intended to be run with root privileges so that it can access as much system information as possible. It is especially useful for system administrators, security analysts, and students learning about digital forensics and incident response.

The project also includes optional AI-generated analysis. After collecting the evidence, the script creates a summary of important findings and sends a sanitized version of that summary to an AI model. The AI model then generates an additional report that may help identify suspicious processes, unusual network connections, indicators of compromise, and the severity of the incident.

Technical Implementation:

The project was built primarily using Bash scripting because Bash is available on most Linux systems and provides direct access to system commands that are commonly used during forensic investigations. Bash also allowed us to automate the collection process without requiring additional programming languages or a complicated setup.

The main script is named `ir_collector.sh`. When the script runs, it creates an evidence directory using the hostname and timestamp of the machine. This helps ensure that every collection is unique and supports chain of custody by recording when the evidence was gathered.

System Information Collection:

The script collects basic system information and saves it to `system_info.txt`. It uses commands such as:

```
uname -a
```

```
cat /etc/os-release
```

```
uptime
```

```
lscpu
```

```
free -h
```

```
df -h
```

This information helps investigators understand the environment they are analyzing.

Process Collection:

The script collects running processes using:

```
ps aux
```

The output is stored in `processes.txt`. This allows investigators to review what programs were running at the time of collection and identify suspicious or malicious processes.

Open File Collection:

The script uses:

```
lsof
```

The output is stored in `open_files.txt`. This can help identify files being used by suspicious processes or applications.

Network Information Collection:

The script collects network information using:

```
ss -tulnp -n
```

```
ip route
```

```
ip neigh
```

This data is stored in network.txt and can help identify suspicious ports, unknown remote connections, and unusual network activity.

Log Collection:

The script copies system log files from /var/log and saves them into a logs/ folder. It also runs:
journalctl --no-pager

These logs can help investigators determine what activity occurred before, during, and after a security event.

Hashing and Data Integrity:

One of the main forensic principles used in this project is data integrity. The script uses SHA-256 hashing to generate hashes for all collected evidence files.

sha256sum

The hashes are stored in hashes.sha256. This ensures that investigators can later verify that the evidence was not modified after collection.

Logging and Chain of Custody:

The script creates a collection.log file that records each major step of the evidence collection process along with timestamps. This supports chain of custody because investigators can see exactly when evidence was collected and what actions were performed.

AI Integration:

The project uses curl and jq to send a sanitized summary of the evidence to an AI endpoint hosted at sushi.it.ilstu.edu. Before sending the information, the script removes sensitive process path data and limits the amount of text being sent.

The AI is prompted to identify:

- Suspicious processes
- Network anomalies
- Indicators of compromise
- Severity level

The AI-generated results are saved to ai_summary.txt. The raw API response is stored in api_raw.json, and the sanitized data sent to the AI is stored in debug_sent_to_ai.txt.

Compression and Encryption:

After evidence collection is complete, the script compresses the evidence folder into a .tar.gz archive and encrypts it using OpenSSL with AES-256-CBC encryption.

```
openssl enc -aes-256-cbc -salt -in archive.tar.gz -out archive.tar.gz.enc
```

The encrypted file is saved using the following format:

```
IR_Evidence_<hostname>_<timestamp>.tar.gz.enc
```

Dependencies:

The script requires several Linux tools and packages:

```
sudo apt update
```

```
sudo apt install -y lsof net-tools iproute2 procps util-linux curl jq openssl
```

Development Process:

The development and testing of this project were performed using a single virtual machine environment. This approach ensured consistency across all stages of development, including scripting, debugging, and validation of outputs. By using one VM, the team was able to maintain a controlled environment where system state, installed dependencies, and permissions were predictable and reproducible.

All features of the script, including system data collection, hashing, logging, compression, encryption, and AI integration, were tested within this VM. Running the script with root privileges allowed full visibility into system processes, logs, and network activity, which was necessary to verify that each component of the tool functioned correctly.

The use of a single VM also helped streamline collaboration between team members, as everyone worked within the same configuration and could replicate results without environment-related inconsistencies. This supported the project goals of repeatability and reliability in forensic evidence collection.

Results:

The final result of the project is a fully automated evidence collection tool that creates an organized forensic evidence package.

When the script runs successfully, it creates a folder similar to the following:

IR_Evidence_Hostname_2026-04-21_12-30-45/

Inside the folder are the following files:

IR_Evidence_Hostname_2026-04-21_12-30-45/

system_info.txt

processes.txt

open_files.txt

network.txt

logs/

summary.txt

ai_summary.txt

hashes.sha256

api_raw.json

debug_sent_to_ai.txt

collection.log

The script then creates a final encrypted archive:

IR_Evidence_Hostname_2026-04-21_12-30-45.tar.gz.enc

The output provides investigators with a complete snapshot of the Linux system at the time of collection. The summary.txt file provides a quick overview of top CPU-consuming processes, active network connections, and recent logs. The AI-generated report provides an additional layer of analysis that can help investigators identify suspicious activity more quickly.

Some examples of suspicious findings that the script could help identify include:

- Unusual processes consuming high CPU resources
- Unexpected listening ports
- Connections to unknown IP addresses
- Recently modified log entries
- Unauthorized user activity
- Persistence mechanisms running in the background

The tool was tested on Linux systems using Ubuntu and Debian-based environments. Running the script as root provided the most complete evidence collection results.

Lessons Learned and Conclusion:

One of the biggest lessons learned during this project was the importance of automation in digital forensics and incident response. Manually collecting processes, logs, network information, and system details takes a long time and increases the risk of missing important evidence. By automating the process, we were able to make evidence collection faster, more repeatable, and more reliable.

We also learned that preserving forensic integrity is extremely important. Hashing evidence with SHA-256 and logging each collection step helped ensure that evidence could be verified later and that the process could be documented properly.

Another lesson learned was that AI can be useful in helping summarize forensic findings. The AI-generated analysis can quickly point out suspicious behavior and provide a higher-level overview of the collected data. However, AI should not be trusted completely on its own. Investigators still need to manually review the evidence because AI may miss details or incorrectly classify certain activity.

One challenge we encountered was making sure the script would work even if some tools were missing from the system. For example, some systems may not have `lsuf` or `journalctl` available. To solve this, the script includes error handling so that missing tools do not stop the entire evidence collection process.

Another challenge was balancing detailed evidence collection with privacy concerns. Since some system data may contain sensitive information, the script sanitizes certain process paths before sending information to the AI endpoint.

If we had more time, we would improve the project by adding:

- JSON output for SIEM integration
- A minimal footprint mode that avoids disk writes until the final archive is created
- A live versus dead system comparison mode
- Better filtering for suspicious processes and connections
- Support for additional Linux distributions
- A graphical user interface

Overall, this project successfully demonstrated how Bash scripting can be used to automate incident response and forensic evidence collection on Linux systems. The final tool provides a

practical and realistic example of how system administrators and security analysts can collect evidence quickly while maintaining important forensic principles such as integrity, repeatability, and chain of custody.